Product Requirements Document (PRD)

POC: AI Voice Agent with Twilio Conversation Relay, Real-Time Conversational Intelligence & Warm Transfer

1. Document Overview & Objective

This document outlines the product and technical requirements for building a local Proof of Concept (POC) demonstrating a state-of-the-art AI-driven customer service line.
The application utilizes **Twilio Conversation Relay** to interface a phone caller with an LLM (OpenAI API). Simultaneously, **Twilio Conversational Intelligence** runs in the background to transcribe the call and generate real-time automated summaries.
When the user requests a human agent, the AI seamlessly triggers a **Conference-based Warm Transfer** to a live PSTN agent. Prior to joining the agent to the call, the system whispers the AI-generated summary to the agent, providing instant customer context.

2. Core Technical Architecture & Tech Stack

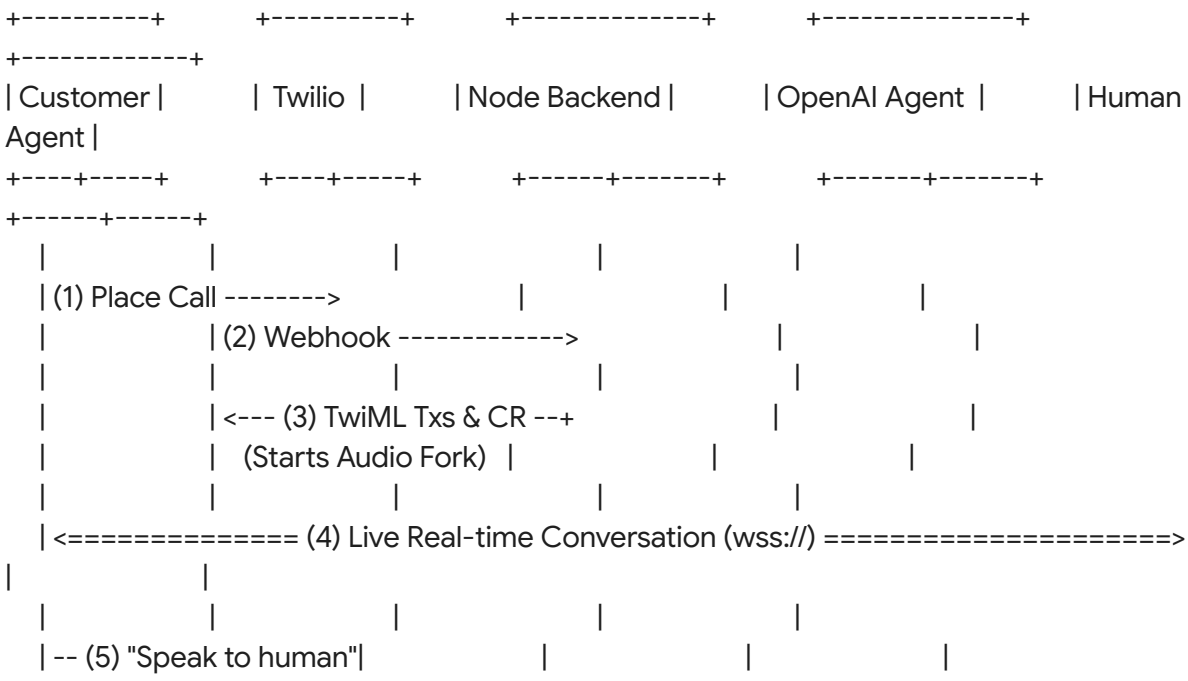
The POC will be built locally using a lightweight, high-performance stack designed for low latency and ease of deployment:

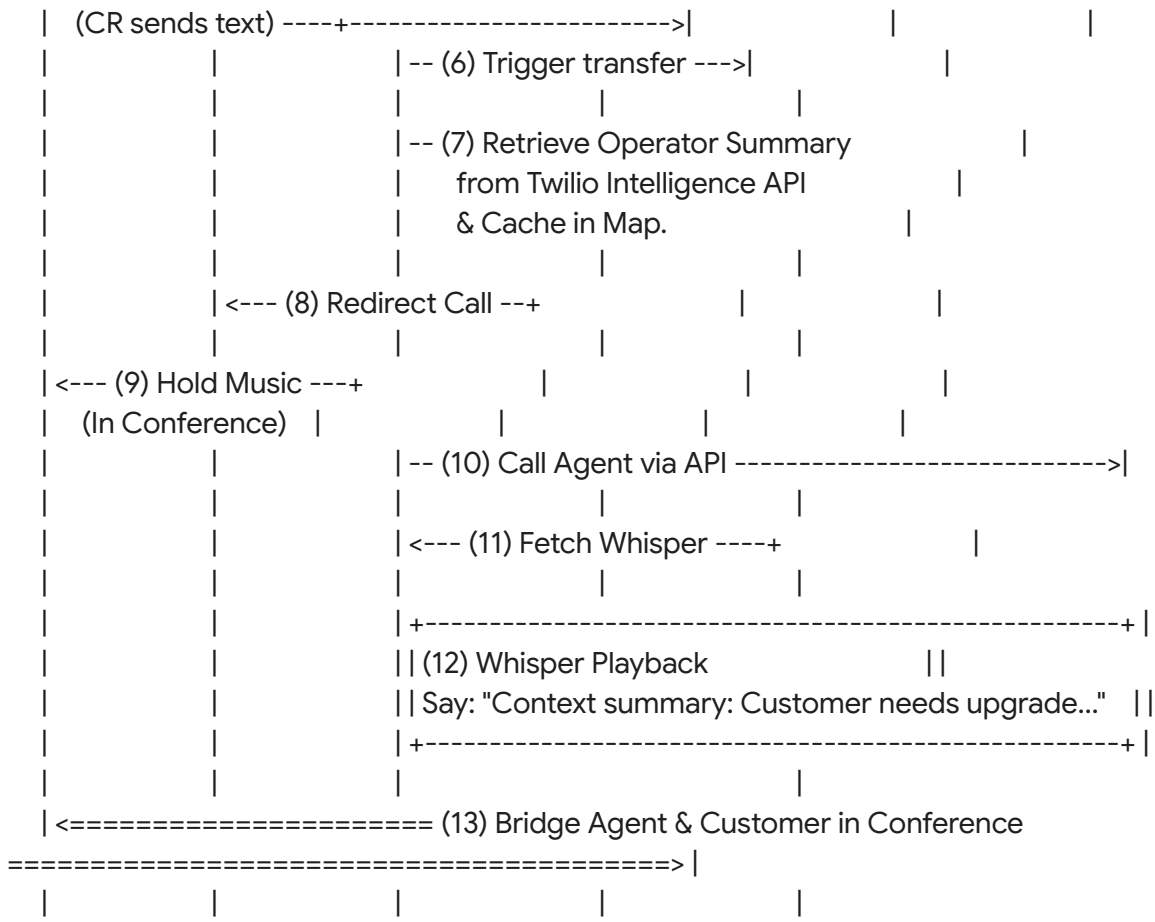
Component	Technology	Role / Purpose
Backend Runtime	Node.js (Express / ws library)	App server orchestrating webhooks, WebSockets, and Twilio REST APIs.
Local Tunneling	**ngrok**	Exposes localhost http:// and ws:// endpoints to the public internet securely with SSL (https:// and wss://).
AI Orchestration	**OpenAI API**	Handles natural language understanding, maintains conversational state, and generates voice agent

		responses.
Transcriber & Summarizer	**Twilio Conversational Intelligence (v3)**	Real-time audio parsing and execution of the GenAI **Summary Operator** (intelligence_operator_01kcv35pnkeysaf6z6cqtbpegn).
PSTN / Voice Carrier	**Twilio Programmable Voice**	Handles inbound call routing, outbound dialing, whispers, and conference bridging.
State Management	**Node.js Memory Map**	In-memory key-value store mapping Call SIDs to Conference Rooms, Agent SIDs, and cached summaries.

3. End-to-End User Flow & Sequence

Below is the step-by-step lifecycle of a call, from initial greeting to warm transfer and fallback.





4. Functional & Feature Requirements

4.1 Inbound Handling & AI Agent Setup

- **Call Entrance:** When a call lands on the Twilio phone number, the backend must return TwiML containing two concurrent instructions:

1. <Start><Transcription>: Initiates live streaming of the call to the configured **Twilio Conversational Intelligence Service**.
2. <Connect><ConversationRelay>: Connects the call's audio stream directly to the local application's secure WebSocket endpoint (wss://).

- **AI Chat Session:** The backend initiates an OpenAI completion session. It handles the low-latency audio/text conversion of conversation turns.

4.2 Intent Detection (PSTN Transfer)

- **Keyword Matching:** The backend websocket parser must actively analyze incoming text transcriptions sent via the Conversation Relay WebSocket.
- **Trigger:** If the user's processed text contains the explicit phrase **"Let me speak to a human"** (or contextual variations like *"speak to a manager"* or *"transfer me"*), the transfer sequence is immediately initiated.

4.3 Real-Time Summary Generation

- **Twilio Language Operator:** The app must hook into Twilio's native Conversational Intelligence API.
- **API Fetch:** At the exact moment the transfer is triggered, the backend queries the OperatorResults resource for the active call's SID to pull down the dynamic GPT-4o-mini-powered transaction summary.
- **Fallback Summary:** If the Conversational Intelligence API has not finished processing the latest summary, the backend falls back to a quick completion request via OpenAI ("Summarize this conversation transcript in 20 words or less...").

4.4 Conference Bridging & Warm Transfer Execution

- **Step A (Hold Customer):** Execute an asynchronous REST API update to redirect the customer's CallSid away from <ConversationRelay> and into a specific TwiML <Conference> room (e.g., room_<CustomerCallSid>). Standard Twilio hold music must play.
- **Step B (Call Agent):** Execute an outbound REST API call to the agent's PSTN number (defined as AGENT_PSTN_NUMBER in .env).
- **Step C (Whisper Context):** The outbound call's routing URL must point to /agent-whisper. When the agent answers, the backend fetches the stored summary from the local Node.js Map and reads it to the agent using <Say>.
- **Step D (Join Room):** Immediately following the <Say> block, the agent's TwiML feeds them into the same <Conference> room, seamlessly merging both tracks.

4.5 Fallback (Human Agent Unavailable)

- **Status Monitoring:** The app must register a statusCallback endpoint for the outbound Agent call.
- **Trigger Conditions:** If the agent's call returns a status of busy, no-answer, failed, or remains unanswered after a timeout threshold (e.g., 20 seconds).
- **Execution:** The backend identifies the corresponding customer's call in the local Map, redirects them out of the <Conference> room, plays the announcement: *"All lines are currently busy, we will call you back as soon as possible,"* and terminates the call safely.

5. System API Contracts & Code Blocks

5.1 Inbound Call TwiML (Initialization)

Exposes endpoint /voice/incoming (POST).

```xml
<Response>
  <Start>
    <Transcription intelligenceService="GAXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX" />
  </Start>
  <Connect>
    <ConversationRelay url="wss://YOUR_NGROK_SUBDOMAIN.ngrok-free.app/relay" />
  </Connect>
</Response>
```

5.2 Retrieving the Summary from Conversational Intelligence

Endpoint invoked by your orchestrator when a transfer is triggered:

```javascript
// Example Node.js snippet fetching the summary from Conversational Intelligence
const fetch = require('node-fetch');

async function getLiveSummary(callSid) {
 const accountSid = process.env.TWILIO_ACCOUNT_SID;
 const authToken = process.env.TWILIO_AUTH_TOKEN;

 const response = await fetch(
  `https://intelligence.twilio.com/v3/OperatorResults?CallSid=${callSid}`, {
   method: 'GET',
   headers: {
    'Authorization': 'Basic ' + Buffer.from(accountSid + ":" + authToken).toString('base64')
   }
  }
 );

 const data = await response.json();

 // Parse for the specific Pre-built Summary Operator ID:
 const summaryOperatorId = 'intelligence_operator_01kcv35pnkeysaf6z6cqtbpegn';
 const result = data.results.find(op => op.operatorSid === summaryOperatorId);
```

```
  return result ? result.textOutput : "Customer requesting a general handoff.";
}
```

5.3 Warm Transfer Execution Webhook

Exposes endpoint /agent-whisper (POST).

```
JavaScript
app.post('/agent-whisper', (req, res) => {
  const roomId = req.query.room;

  // Fetch cached summary from local in-memory Map
  const summary = activeSessions.get(roomId)?.summary || "A customer is waiting.";

  const xmlResponse = `
  <Response>
    <Say voice="Polly.Joanna">
      Incoming transfer. Summary of conversation: ${summary}. Transferring you in now.
    </Say>
    <Dial>
      <Conference>${roomId}</Conference>
    </Dial>
  </Response>
  `;
  res.type('text/xml');
  res.send(xmlResponse);
});
```

6. Local Project Configuration & Setup

To run this POC locally, the project must utilize standard configurations matching this template:

File Structure:

```
Plaintext
twilio-voice-poc/
├── .env
├── package.json
└── server.js
```

Required .env File Template:

```ini, TOML
PORT=3000
TWILIO_ACCOUNT_SID=ACXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
TWILIO_AUTH_TOKEN=your_twilio_auth_token
TWILIO_PHONE_NUMBER=+15550001111
AGENT_PSTN_NUMBER=+15552223333 # Variable to target dynamic human agent
OPENAI_API_KEY=sk-proj-...
TWILIO_INTEL_SERVICE_SID=GAXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

State Management Structure (Node.js Map):

```JavaScript
// Key: Conference Room ID (or Customer CallSid)
// Value: Session Metadata Object
const activeSessions = new Map();

/* Example state structure:
activeSessions.set("room_CA123", {
  customerCallSid: "CA123",
  agentCallSid: "CA456",
  summary: "Caller encountered checkout billing issue.",
  status: "transferring"
});
*/
```

7. Success & Testing Criteria

For a successful local verification of the POC, the system must pass the following manual checks:

1. **Interactive AI Chat:** Call your Twilio phone number; the AI agent must respond promptly using OpenAI context.
2. **Dynamic Triggering:** Saying *"Let me speak to a human"* must immediately stop the AI agent voice, put the customer on hold music, and dial the configured AGENT_PSTN_NUMBER.
3. **Whisper Validation:** The human agent must pick up their phone and hear the specific AI-generated conversation summary read aloud.
4. **Active Bridge:** Upon completion of the whisper, the agent and customer must be

bridged together seamlessly with low latency.

5. **No-Answer Fallback:** Triggering a transfer while leaving the agent's phone offline must result in the customer hearing: *"All lines are currently busy, we will call you back as soon as possible"* before cleanly disconnecting.